

Prediction of Road Traffic Flow Based on Deep Recurrent Neural Networks

Zoe Bartlett and Liangxiu Han

School of Computing, Mathematics and Digital Technology
Manchester Metropolitan University
M1 5GD, UK

Email: z.bartlett@mmu.ac.uk and l.han@mmu.ac.uk

Trung Thanh Nguyen and Princy Johnson

Faculty of Engineering and Technology
Liverpool John Moores University
L3 5UA, UK

Email: t.t.nguyen@ljmu.ac.uk and p.johnson@ljmu.ac.uk

Abstract—Traffic congestion is a major issue for developed countries; therefore, research into the prediction of road traffic flow is vital. Deep neural networks, such as deep recurrent neural networks, are now being explored for road traffic flow prediction. However, what deep architecture is the most appropriate remains unanswered. Previous research into deep recurrent neural networks fails to compare them to other deep models; instead, comparisons are made with simple shallow models. To compound this issue, standard performance metrics assess a model's success solely on its accuracy. No consideration is given to computational cost. Furthermore, optimisation of a neural network's architecture can be difficult. There is no standard or analytical method to determine their correct structure. This often leads to sub-optimal architectures being used. Therefore, deep neural networks should be assessed on how sensitive the model is to architectural changes. In this paper, we have examined three recurrent neural networks (a standard recurrent, a long short-term memory, and a gated recurrent unit) to determine how they perform on time-series data based on a real dataset. We compared their accuracy, training time, and sensitivity to architectural change. Additionally, we developed a new performance metrics, Standardised Accuracy and Time Score (STATS), which standardises the accuracy and training time into a comparable score, allowing an overall score to be awarded. The experimental results show that, based on the STATS, the gated recurrent unit produced the highest overall performance and accuracy score. Furthermore, its prediction was most stable against architectural changes. Conversely, the long short-term memory was the least stable model.

Index Terms—Machine Learning, Short-term Traffic Prediction, Deep Learning, Deep Neural Networks (DNN), Recurrent Neural Network (RNN), Long Short-Term Memory Neural Network (LSTM), Gated Recurrent Unit Neural Network (GRU), Artificial Neural Networks (ANN), Performance Metric

I. INTRODUCTION

In many developed countries traffic congestion is a major issue. Research in the last 20 years has concentrated on building models to develop an Intelligent Transport System (ITS). Past research includes a variety of machine learning models, such as K Nearest Neighbours and Support Vector Machines, however, multiple studies have shown that non-parametric models like neural networks perform better at predicting complex time-series data, such as road traffic flow [1]. Furthermore, due to advances in computing power and algorithm development by Hinton et al. [2], the depth of neural networks is increasing, leading to superior perfor-

mances. Deep neural networks (DNNs) [3] are now feasible and more efficient for large complex data [4]. They are favoured over shallow learners due to their ability to efficiently extract complex latent patterns embedded within the data [5] owing to their long computational chain of layers [6]. Despite this, neural networks designed for road traffic prediction are predominately shallow learners with only one hidden layer [7]. Therefore, research into exploring deep architectures to improve prediction accuracy for road traffic flow is now possible and needed.

There is a wealth of neural network architectures and hybrids. The real challenge in the DNN domain is model selection [8]. What deep architecture is the most appropriate for noisy time-series data, such as road traffic flow, remains unanswered [9]. To compound this issue more, there is a lack of research comparing deep learning models for road traffic flow prediction. Researcher's who do make comparisons often compare their models to simple shallow architectures or statistical models which can not compete with the long computational chain of layers of DNN. Another major flaw of DNN research is its disregard for computational cost (training time). It is obvious that the deeper a neural network, the bigger the computational cost, however, this is often omitted from evaluations. Traffic flow prediction models assess success solely on prediction accuracy; predominately the root mean squared error (RMSE) or mean absolute percentage error (MAPE) is used to evaluate a model [10]. No evaluation of the training time taken to produce the prediction is made. Additionally, no trade-off between the accuracy and training time is investigated. The current standard performance metric is antiquated. Successful models should produce tolerably accurate results within a reasonable time frame set by the specific problem domain. Therefore, a new performance metric is needed for DNNs.

Furthermore, to fairly compare different DNNs each model should be fully optimised to achieve its best prediction, yet there is no standard procedure or analytical calculation to determine the optimal architectural structure. Hyperparameters are optimised using domain knowledge through experience and literature review, or heuristics through grid search style algorithms. Many model optimisations are difficult, not consistent, and at times not appropriate for the domain problem.

Therefore, assessing how sensitive a model is to architectural change is important. It should be established how consistently a model produces an acceptable prediction within a tolerable time frame set by the domain problem.

The contributions of this work include: 1) we have examined and compared three existing deep recurrent neural networks (a basic recurrent, a long short-term memory, and a gated recurrent unit), using a novel performance metric, based on a real dataset to determine which is suitable for road traffic prediction, 2) for fair comparison, a new performance metrics, the standardised accuracy and time score (STATS), was developed to assess the overall performance of the DNNs based on their prediction accuracy and training time, and 3) we have examined the DNNs' sensitivity to architectural changes.

The rest of this paper is organised as follows: Section II presents the State of the Art in Deep Learning Traffic Flow Prediction Models; Section III describes the Methodology used for experimentation; Section IV details the Experimental Evaluation; and Section V discusses the Conclusion and Future Work.

II. STATE OF THE ART IN DEEP LEARNING TRAFFIC FLOW PREDICTION MODELS

In this section, we will review and access deep recurrent architecture for their suitability for the prediction of road traffic flow.

Lv et al. in 2014 [7] is, to the best of our knowledge, the first publication on DNNs for road traffic prediction. Lv et al. stated that shallow prediction models with limited hidden nodes do not learn an adequate representation of the relationship between the current and the future traffic flow due to the data's stochastic and complex non-linear properties. Instead, a compressed version is found. Thus, an DNN is needed; however, they do have limitations. Adding unneeded layers can negatively affect the model's prediction accuracy and increase training time. More et al. in 2016 [11] used traffic flow data from Dublin, Ireland, with a five-minute time-step and a Jordan neural network model (JNN) [12] to predict road traffic flow. A JNN is an adaptation of an RNN that can be identified by the inclusion of a *context unit*. The context unit receives the previous time-steps prediction from the output layer (y_{t-1}) to provide *context* for the current prediction (RNNs receive context from the previous time-step's hidden state (h_{t-1})). More et al. concluded that the structure of an ANN determined how successful a prediction was, and reported "almost 98% accuracy for most of the datasets" [11]. However, the study failed to compare the model to any other ANNs. Other variations of RNNs may be more suitable. State of the art research using RNNs for time-series prediction have focused on other adaptations of RNNs due to the basic structure of RNN's and JNN's being unable to capture the long-term dependencies within time-series data [13]. An RNN can not learn dependencies if the time lag is greater than 5-10 discrete time-steps apart [14]. This severely limits the range of contextual information of an RNN [15]. Furthermore, the success of an RNN is hindered by vanishing gradient

problems [16]. Therefore, does an RNN have any advantage over FFNNs?

Long short-term memory (LSTM) neural networks [17] were initially developed to solve the *vanishing/exploding gradient problem* through the introduction of a *memory* called the *cell* and the use of a *constant error carousel* (CEC). An CEC denotes the recurrent connection of the *cell state* (c_t), where an error can flow through *unchanged*, as depicted in Figure 1 by the directed circle encompassing the word 'cell'. Therefore, in an LSTM an error is reduced when it is backpropagated through the multiple layers, however, it is not modified in the CEC as it is designed to preserve the error through time and layers. Preserving the error allows the model to continue to learn over many time-steps, therefore, link cause and effects which are many time-steps apart.

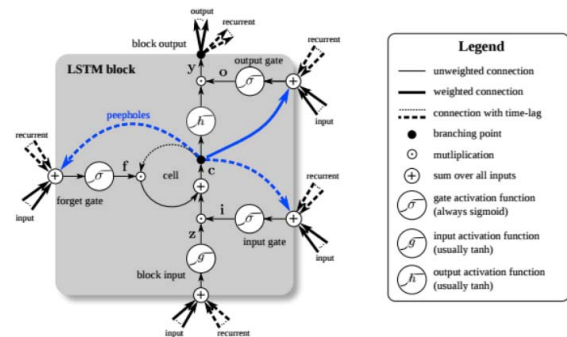


Fig. 1. A detailed schematic of a long short-term memory block (also known as a cell) [18]

Zhao et al. in 2017 [19] used an LSTM for the prediction of road traffic flow using temporal-spatial correlations. The traffic data used was 19 days, split into five-minute time-steps, of 500 observation points on one road segment (although only three points were used for prediction). The paper pre-processed the traffic data using an *origin-destination cost* (ODC) matrix to find the correlations between the temporal and spatial data. Once the data is preprocessed it passed through a standard LSTM. The model was compared using four different LSTMs with two, three, five, and six layers to find its optimal structure. It was also compared an ARIMA, SVM, RBF, SAE, and a standard RNN. The paper found that preprocessing the input data improved accuracy and the LSTM was the most accurate. However, the trade-off between training time and accuracy was not considered. Is the additional computational time to preprocess the data justified by the increase in accuracy? Furthermore, it is not explicitly stated if the correlation data was also used in the comparison models.

Shi et al. in 2018 [20] used an LSTM to predict household loads. Shi et al. stated that the main problem with DNNs are their tendency to overfit to training data and therefore, results can not be generalised. Furthermore, the input data, household loads, is noisy due to external factors. To overcome these problems Shi et al. preprocessed the input data in a *pooling layer*. The pooling layer is designed to compensate

for the small amount of temporal data by adding a new input parameter to the target household. The new parameter added is the neighbour's household load. By adding neighbour's household load the input data has more diversity. The training data from 929 households divided into pools of ten were used, which consisted of 48 hours of house loads with a time-step of 30 minutes. The pooled training data was passed through LSTM and its predictions was compared with an ARIMA, SVR, and an RNN. It found that the model's accuracy, in terms of RMSE, was superior. The main issue with the paper is the problem it seeks to overcome, the overfitting of DNNs. If a model is overfitting the regularisation term needs to be adjusted, or other methods, such as dropout rates and early stopping of training, can be used. The paper has failed to mention these methods or address their inadequacies and state why they have chosen to develop a new method. The paper also fails to address the increase in training time due to the addition of a pooling layer.

In conclusion, although the internal memory of an LSTM has proven it can remember data over many time lags it is yet to be demonstrated that they can perform complex rational reasoning with this information. Many models still adjust their input data to provide more information to aid the models' prediction accuracy. Furthermore, due to their complex cell structure, as seen in Figure 1, the training of an LSTM has high computational costs. No paper has addressed the trade-off between the training time and accuracy.

Cho et al. in 2014 [21] put forward another adaptation of an RNN to solve the vanishing gradient problem, a gated recurrent unit (GRU) neural network. Similar to the LSTM, the GRU can be trained to retain information over many time lags through the use of gates. GRUs are still in their infancy, therefore, there is limited research regarding them, with most papers performing comparative studies. Fu et al. in 2016 [9] conducted a comparison study of an ARIMA, LSTM, and an GRU. The study used four weeks of data from 50 sensors in Oakland, USA with a training and test split of three weeks and one week respectively, and a time-step of five-minutes. Fu et al. found that the GRU model was, in terms of mean squared error (MSE), most accurate for prediction road traffic flow. However, no other architectures of DNNs were compared, and the models' training times were not considered.

Similarly, Zhang and Kabuka in 2018 also performed a comparison study using the same data as the paper above, but also included weather data such as wind speed, precipitation, and temperature. The study compared an ARIMA, a support vector machine, an RNN, LSTM, and an GRU model. The study found that, in terms of RMSE, the GRU model was the most accurate. Furthermore, the model was compared with and without the weather data and discovered that its inclusion improved the model's prediction by 25%. Again, the paper only compares the accuracy of the neural network architectures. No training time was considered. Furthermore, the models' sensitivity to architectural change was not investigated. The GRU model may have produced the most accurate prediction, however, it is not know how volatile its predictions are.

In conclusion, RNNs are designed for time-series prediction and have provided good prediction results. However, there is still a lack of research comparing deep learning models. Many comparisons are still made with simple shallow architectures which can not compete with DNNs long chain of computational layers [20]. Furthermore, common adaptation of RNNs tend to focus on how best to preprocess and include different input parameters to improve the model's accuracy. No consideration is given to the trade-off between the accuracy and training time. Therefore, the real challenge of DNNs still exists, the model selection [8]. What deep architecture is the most appropriate for noisy time series data such as road traffic flow remains unanswered [9].

III. METHODOLOGY

The main aim of this paper is to accurately predict the road traffic flow, on a typical busy urbanised arterial road, with a time horizon of five-minutes. In order to achieve this aim we have focused on three research questions: 1) what is the most suitable deep RNN for road traffic flow prediction based on its accuracy and training time, 2) what is the most appropriate performance metric for deep neural networks, and 3) which deep RNN is the least sensitive to architectural change (the model's range of predictions over various architectural structures)?

To answer all three research questions 36 experiments were conducted utilising three deep recurrent learning models (12 per model) using different architectural setups and a real dataset. The prediction accuracy and training time were compared and analysed.

A. Data Description

The deep learning models were applied to a real-life dataset from an arterial road between Manchester and Liverpool, UK. The dataset consisted of three months of data collected between 1st January to 31st March 2016, with a time horizon of five-minutes (26,195 data point) and input parameters of different vehicle classes as shown in Table I. Two months of data (18,337 data points) was used for training and validating the models, and one month of data (7,858 data points) was used for testing the models.

TABLE I
CLASSES OF VEHICLE TYPE

Class No.	Vehicle Type
1	Motorcycles
2	Car or Van
3	Car or Van with Trailer
4	Rigid Goods
5	Articulated HGV
6	Bus or Coach

B. The Model Architectures

The deep recurrent learning models chosen for comparison and used during experimentation were:

- 1) An RNN model

- 2) An LSTM model
- 3) An GRU model

All models were setup using 12 different architecture structures consisting of two, three, five, and six layers (excluding any input and output layer) [19] with 12, 16, and 20 nodes in each layer to find the optimal structure. All weights and biases were randomly initialised, and the activation function used was the sigmoid activation function as per Zhao et al. setup in 2017 [19]. A drop out rate of 50% was used after every layer to stop the models from overfitting. All other hyperparameters were determined by grid search, and for standardisation all the deep learning models were trained the using Adamax optimiser [23]. Below each model is explained in detail.

1) *RNN Model*: A RNN is designed to detect sequences in temporal data by the assuming the input and output pairs are dependent on each other, meaning the model takes into account the previous prediction when calculating the current prediction. Therefore, the order of the sequence is taken into consideration; the temporal element of the input data adds a new dimension to the prediction function being learned. The model achieves this through the use of *loops*. RNNs contain a *hidden state node* (s) that feeds back the previous time-step ($t-1$) value. This allows the model to find and use correlations between the temporal data that may be many time-steps apart, as seen in Figure 1, and is often referred to as an *internal memory*. Therefore, the current hidden state is a function of current input data (x_t) and the previous time-step's (h_{t-1}) hidden state, as seen in Equation 1, where h_t is the hidden state, t relate to the time-step, σ is the activation function, x_t is the input data at time t , and u is the weight matrix.

$$h_t = \sigma_h(u_h x_t + w h_{t-1} + b_h) \quad (1)$$

Therefore, the prediction (y_t) is calculated as shown in Equation 2.

$$y_t = \sigma_y(w_y h_t + b_y) \quad (2)$$

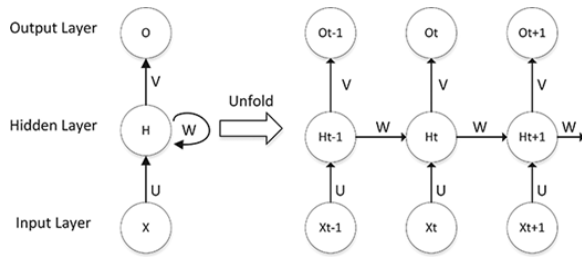


Fig. 2. A basic RNN with its hidden state unfolded [24].

2) *LSTM Model*: A standard LSTM NN *cell* contains three *gates*, a *forget gate* (f), an *input gate* (i), and an *output gate* (o). It contains a hidden state (h) and a *memory state* (C), as seen in Figure 1.

The memory state is the main concept behind the LSTM NN; it transfers the data through the cell via a sequence of gates. The gates decide what information in the cell state

should be added or removed. Each gate is a single-layered neural network with a sigmoid activation function (σ). The sigmoid function outputs a value between zero and one (see Equation 3) to determine how much or how little should be added or deleted from the cell state.

$$\sigma(x) = \frac{e^x}{e^x + 1} \quad (3)$$

The cell state is first computed by calculating the block input (z) based on the previous hidden state (h_{t-1}) and the current input data (x_t), as shown in Equation 4, where w is a weight and b is a bias. An activation function, usually $\tanh$ (as shown in Equation 5), is used to regulate the network by squashing the input values between -1 to 1.

$$z = \tanh(w_{xz}x_t + w_{hz}h_{t-1} + b_z) \quad (4)$$

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5)$$

At the same time the input gate is calculated. This gate decides what should be added to the cell state (c_t) from the current input (x_t), the previous hidden state (h_{t-1}), and the previous cell state (c_{t-1}), as shown in Equation 6.

$$i = \sigma(w_{xi}x_t + w_{ci}c_{t-1} + b_i) \quad (6)$$

The sigma activation function of the input gate (i) outputs a value between zero and one. Therefore, when the block input and input gate are multiplied together, only the values the input gate has determined important are passed through. The current cell state can be calculated as $z \times i$.

The second gate calculated is the forget gate. The forget gate decides what part of the cell state should be allowed to pass through and what should be forgotten from the current input (x_t) and the previous hidden state (h_{t-1}), as shown in Equation 7. The output of the forget gate enters the CEC and is multiplied by previous cell state (c_{t-1}), then added to the output of the previous step, block input and input gate, as shown in Equation 8.

$$f = \sigma(w_{xf}x_t + w_{cf}c_{t-1} + b_f) \quad (7)$$

$$c_t = c_{t-1} \circ f_t + z \circ i \quad (8)$$

Gers et al. would go on to add *peepholes* to the architecture in 2000 [14] which allows the gates to connect to the CEC. Therefore, the hidden state (h_t) in Equations 6, 7, and 9 has been replaced with cell state (c_t).

Lastly, the output gate is calculated. The output gate decides how much of the new memory should be passed through to the output from the cell based on the previous hidden state (h_{t-1}) and the current input data (x_t), as shown in Equation 9. Again, an activation function of sigmoid is applied to squash the values between zero and one.

$$o = \sigma(w_{xo}x_t + w_{co}c_t + b_o) \quad (9)$$

The cell state is then passed through an activation function, usually $\tanh$, to regulate its values between minus one and one, then multiplied by the output gate's output to determine the

output of the cell, and what will be passed forward as shown in Equation 10.

$$h_t = \tanh(c_t) \circ o_t \quad (10)$$

3) *GRU Model*: The gates included in a standard GRU cell are an *update gate* and a *forget gate*, as shown in Figure 3. The current input (x_t) and the previous hidden state (h_{t-1}) is

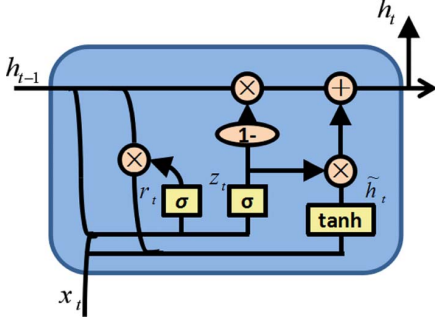


Fig. 3. A gated recurrent cell [21]

added together and passes through the update gate, as shown in Equation 11. The update gate decides what information should be forgotten and what should be added. A sigmoid activation function (as shown in Equation 3) is used to squash the values of the input between zero and one.

$$u = \sigma(w_{xu}x_t + w_{hu}h_{t-1} + b_u) \quad (11)$$

Next, the same input (x_t and h_{t-1}) is passed through the reset gate with a sigmoid activation function (as shown in Equation 12). The reset gate is used to decide how much of the past information should be forgotten.

$$r = \sigma(w_{xr}x_t + w_{hr}h_{t-1} + b_r) \quad (12)$$

The hidden state (h) is then updated using the reset gate and the current input (x_t) (as shown in Equation 13), where the product of the reset gate (r_t) and the weighted previous hidden state ($w_{hh}h_{t-1}$) is the *Hadamard* product.

$$h_t = \tanh(w_{xh}x_t + (1 - r_t) \circ w_{hh}h_{t-1} + b_h) \quad (13)$$

Finally, the hidden state is updated using the update gate to determine what information from the current memory should be stored, as shown in Equation 14.

$$h_t = z_t \circ h_{t-1} + (1 - z_t) \circ h_t \quad (14)$$

C. Performance Metrics

In order to evaluate and compare the accuracy of the models the Root Mean Squared Error (RMSE), as shown in Equation 15, was used to measure the average deviation between the predicted value and the actual value.

$$RMSE = \sqrt{\frac{\sum_{t=1}^n (F_t - A_t)^2}{n}} \quad (15)$$

F_t is the predicted value and A is the actual value at time t , and n is the number of time-steps predicted.

The RMSE is a standard performance metric for the evaluation of time-series prediction models; it is also the most commonly used in road traffic flow prediction [10]. However, the fundamental problem with this performance metric, and its counterparts such as mean absolute percentage error, is they only evaluate the accuracy of a model. No consideration is given to the training time, or the trade-off between accuracy and training time. This is not appropriate for DNNs. As technology advances, so does the depth and breadth of DNNs that are feasible. However, this advancements comes at a price, the training time. DNNs must be assessed on their overall performance, accuracy, and training time. To compound the issue further, limited studies that do consider training time find it challenging to compare the accuracy and training time due to both results being in different units of measurements. Therefore, performance metrics like RMSE for DNNs are antiquated.

We propose a novel performance metric, the **Standardised Accuracy and Time Score (STATS)**, as shown in Equation 18. STATS overcomes the limitations of the standard performance metric by evaluating both the accuracy and the training time of a model. Furthermore, these evaluations are standardised in a set range (0 to 100, where the higher the score the better the performance) allowing comparisons to be made. Additionally, due to the scores being standardised they can be combined (accuracy and training time) to give a model an overall performance score.

The accuracy and training time scores are normalised by their ratio to the range of the results, as shown in Equation 16. The range can be adapted for specific time-series problems through the use of upper and lower bounds, as shown in Equation 17 to 19. The bounds may be used to disregard any accuracy or training times which fall beyond a set range determined by the user. For example, setting an upper bound of 20 for the accuracy score will score any model with an RMSE of 20% or above as zero for accuracy, and vice versa, setting a lower bound on accuracy as 10%, any models achieving 10% or better will be scored as full marks (100%) for accuracy. The same logic extends to the upper and lower bounds of the training times.

$$STATS = \left(\frac{w_a}{\alpha_a - \beta_a} \times (\alpha_a - x_a) \right) + \left(\frac{w_t}{\alpha_t - \beta_t} \times (\alpha_t - x_t) \right) \quad (16)$$

$$x = \begin{cases} U & \text{if } x \geq U, \\ L & \text{if } x \leq L, \\ x & \text{otherwise} \end{cases} \quad (17)$$

$$\alpha = \begin{cases} U & \text{if } x \geq U, \\ x & \text{otherwise} \end{cases} \quad (18)$$

$$\beta = \begin{cases} L & \text{if } x \leq L, \\ x & \text{otherwise} \end{cases} \quad (19)$$

α_a is the largest accuracy (RMSE), β_a is the smallest accuracy (RMSE), α_t is the largest training time, β_t is the smallest

training time, w_a is the weighting for the accuracy score, w_t is the weighting for the training time score (where $w_a + w_t = 100$), x_a is the actual accuracy (RMSE), and x_t is the actual training time. In this paper no upper or lower bounds were used during experimentation, and the accuracy and training time were given equal weighting ($w_a \wedge w_b = 50$).

IV. EXPERIMENTAL EVALUATION

A. Comparison of Existing Deep RNNs

In order to answer research question one and two, what is the most suitable deep RNN for road traffic flow prediction based on its accuracy and training time and what is the most suitable performance metric for DNNs, three deep RNNs were used with the 12 different architectural structures, as described in Section III-B, and applied to the real traffic flow dataset, as illustrated in Section III-A. The models were used to predict the traffic flow at the next time-horizon (five minutes). A comparison of the models was made based on their accuracy using the performance metric RMSE (Equation 15) and their accuracy, training time, and overall score using the novel performance metric, STATS, as shown in Equation 16. The RMSE and STATS results were then compared.

TABLE II
THE COMPARISON OF THE PERFORMANCE SCORES OF DEEP RNN MODELS

Model	RMSE (%)	STATS		
		Accuracy Score (%)	Time Score (%)	Overall Score (%)
RNN	17.32	80.46	98.79	89.62
LSTM	9.71	98.32	82.23	90.27
GRU	9.26	98.59	82.77	90.68

Table II shows the results of the model architecture that provided the best RMSE and the best overall performance score using STATS, with a breakdown of its accuracy and time score, for each deep RNN. In terms of the RMSE, the GRU was the most accurate with an RMSE of 9.26%, closely followed by the LSTM with an RMSE of 9.71%. The RNN model was the least accurate with an RMSE of 17.31%. Based on the RMSE it could be assumed that the GRU and LSTM's performances are of comparable success. Table II also shows that, overall, the GRU was the most successful model for the STATS performance metric. It was the most accurate, with an accuracy score of 98.59%, again, closely followed by the LSTM, with an accuracy score of 98.32%. The RNN was the least accurate model with an accuracy score of 80.46%. However, the STATS also shows that the GRU is faster than the LSTM, with a time score of 82.77% and 82.23% respectively, providing more insight into the model's performance. Therefore, the STATS performance metric was able to provide additional information than the RMSE to allow a more balanced evaluation of the deep learning model, based on both their accuracy and training time. Furthermore, an overall score was awarded to each deep RNN model based on their accuracy and training time (in this experimentation

$w_a \wedge w_b = 50$), to establish which model was most successful overall. Consequently, the STATS performance metric was the most suitable for DNNs.

B. Comparison of Existing Deep RNNs' Sensitivity to Architectural Change

In order to answer research question three, what is the most suitable DNN (RNN) for road traffic flow prediction based on sensitivity to architectural change (the model's range of predictions over various architectural structures), three deep RNN were used with the 12 different architectural structures, as described in Section III-B, and applied to the real traffic flow dataset, as illustrated in Section III-A. The models were used to predict the traffic flow at the next time-horizon (five minutes). A comparison of the models was made based on their accuracy, training time, and overall performance using the novel performance metric, STATS, as shown in Equation 16. Additionally, the STATS results were analysed (range, mean, and standard deviation) to determine which model was most sensitive to architectural change (the model's range of predictions over various architectural structures).

TABLE III
THE STATS PERFORMANCE OF DEEP RNN MODELS

Model	STATS		
	Accuracy Score	Time Score	Overall Score
RNN	80.46	98.79	89.62
LSTM	98.32	82.23	90.27
GRU	98.59	82.77	90.68

Table III shows the results of the model architecture that provided the best STATS for each deep RNN. The RNN was overall the least successful model. Although its time cost score was high at 98.79%, its prediction accuracy was poor, only scoring 80.46%. The GRU performed the best overall, with an overall score of 90.68%, beating the LSTM in both accuracy and time score, scoring 98.59% and 82.77% respectively. Therefore, based on the accuracy and time score, deep GRUs are most appropriate for noisy time-series data like road traffic flow.

TABLE IV
THE PERFORMANCE SCORES OF THE DEEP RNN MODEL

Performance Indicator	STATS		
	Accuracy Score (%)	Time Score (%)	Overall Score (%)
Highest	80.46	100	89.62
Lowest	29.42	68.36	50.46
Mean	55.61	84.55	70.08
Range	51.05	31.61	39.16
Standard Deviation	13.72	12.19	12.12

Table IV shows that the RNN architectures were consistently fast, with the highest mean time score of 84.55%, a small range of 31.61%, and a small standard deviation of 12.19%. This is due to the RNN computational structure, as

shown in Figure 2, which allows the model to be computationally efficient at the price of poor prediction results. The accuracy score was less stable, with a mean score of 55.61%, a range of 51.05%, and a standard deviation of 13.72%. The model's accuracy was sensitive to architectural structure changes. Therefore, the RNN performed the worst overall. The simplistic RNN computational structure is unable to capture the long-term dependencies within time-series data. It is unable to learn dependence between an input and output if the time lag is greater than 5-10 discrete time-steps. This has limited the contextual information that the RNN can use for prediction. The results show that more contextual information from distant time lags is needed to produce successful prediction results. Therefore, a RNN is not suitable for the prediction of road traffic flow.

TABLE V
THE PERFORMANCE SCORES OF THE DEEP LSTM NEURAL NETWORK MODEL

Performance Indicator	STATS		
	Accuracy Score (%)	Time Score (%)	Overall Score (%)
Highest	98.32	83.18	90.57
Lowest	0	0	40.48
Mean	78.28	47.23	62.78
Range	98.31	83.18	49.45
Standard Deviation	27.19	29.25	17.10

Table V shows that the LSTM's accuracy and time score were both very unstable. The model produced the largest range and standard deviation for both accuracy and time score of all the models. Therefore, both the model's accuracy and training time is highly sensitive to architectural structure change. This can be attributed to the intricate structure of an LSTM cell, as shown in Figure 1. The CEC allows the model to link cause and effect over many distant time lags, leading to an improvement in prediction accuracy over the RNN. However, the cell contains multiple gates. Each gate is a single-layer neural network. Multiple neural networks have caused the model's training times to be slow and very sensitive to architectural change. Therefore, using an LSTM is a trade-off between accuracy and training time.

TABLE VI
THE PERFORMANCE SCORES OF THE GRU NEURAL NETWORK MODEL

Performance Indicator	STATS		
	Accuracy Score (%)	Time Score (%)	Overall Score (%)
Highest	100	83.31	90.68
Lowest	55.71	16.55	39.80
Mean	87.22	51.03	69.12
Range	44.29	66.76	50.88
Standard Deviation	13.18	25.22	17.11

Table VI shows that the GRU produced the highest mean accuracy score, with the lowest range and lowest standard

deviation. Therefore, the GRU's accuracy was the least sensitive to structural changes of all the models. Furthermore, the GRU's time score range and standard deviation was larger than the RNN's, however, it was not as sensitive to structural changes as the LSTM, with a lower time score range and standard deviation. Consequently, the GRU was overall the least sensitive to architectural change. The GRU's accuracy is due to the intricacy of its cells, as shown in Figure 3, which contains an internal memory that allows the model to learn dependencies over many time lags, more than a standard RNN. Consequently, the model is able to add more contextual information to its predictions. However, the model is less computationally heavy than an LSTM, by only containing two gates (an input gate and an output gate), resulting in a better time score. Therefore, based on results above, taking into account both the overall performance (accuracy and training time) and the sensitivity to architectural change of the deep RNNs, the GRU was the most successful model.

V. CONCLUSION

In this work, we have performed a comparative analysis of three deep RNN architectures (an RNN, an LSTM, and an GRU) by applying them to a real dataset to determine which was most successful for the prediction of road traffic flow based on their accuracy and training time. We also developed a novel performance metric, STATS, that allowed the comparison of both accuracy and training time by standardising them into a comparable score for the overall evaluation of DNNs. Furthermore, we then compared the deep DNNs' sensitivity to architectural changes to determine how stable the models' prediction accuracy were. The results showed that the GRU model produced the most overall successful prediction, producing good results in both accuracy and training time. Furthermore, the GRU model was the least sensitive to architectural change. Therefore, based on these results it can be concluded that the long-term patterns embedded within the input data are needed for the successful prediction of road traffic flow.

For future work, the long-term patterns embedded within traffic flow data should be explored further. What patterns (daily, weekly, or monthly) are needed for the successful prediction of road traffic flow? Furthermore, the novel performance metric, STATS, should be used to evaluate hybrid DNN models. As stated in the literature review, common adaptations of RNNs focus on how best to preprocess and include different input parameters to improve accuracy. No consideration is given to the trade-off between accuracy and training time of the models. Therefore, future work should include evaluating preprocessing layers used to detect additional features, such as stacked autoencoders or convolutional neural networks. Additional layers can improve prediction results, however, this is at the expense of the training time of the model. Increasing the training time of the model affects the model's overall performance in practical settings. Therefore, are these additional layers adding any real value to the models?

ACKNOWLEDGEMENTS

This research is jointly funded by Manchester Metropolitan University and Liverpool John Moore University. We would also like to thank Transport for Greater Manchester for providing the traffic flow data.

REFERENCES

- [1] Z. Bartlett, L. Han, T.T. Nguyen, and P. Johnson. A Machine Learning Based Approach for the Prediction of Road Traffic Flow on Urbanised Arterial Roads. In *16th International Conference on Smart City 2018*, 2019. ISBN 9781538666142.
- [2] Geoffrey E. Hinton, Simon Osindero, and Yee-Whye Teh. A Fast Learning Algorithm for Deep Belief Nets. *Neural Computation*, 18(7):1527–1554, 7 2006.
- [3] Rina Dechter. Learning While Searching in Constraint-Satisfaction-Problems. (PDF Download Available). In *Proceedings of the 5th National Conference on Artificial Intelligence*, page Volume 1: Science, Philadelphia, PA, 1986. ResearchGate.
- [4] Corinna Cortes. Support-Vector Networks. Technical report, 1995.
- [5] Wenhao Huang, Guojie Song, Haikun Hong, and Kun-qing Xie. Deep Architecture for Traffic Flow Prediction: Deep Belief Networks With Multitask Learning. *IEEE Transactions on Intelligent Transportation Systems*, 15(5):2191–2201, 10 2014.
- [6] Jrgen Schmidhuber. Deep Learning in Neural Networks: An Overview. *Neural Networks*, 61:85–117, 1 2015.
- [7] Yisheng Lv, Yanjie Duan, Wenwen Kang, Zhengxi Li, and Fei-Yue Wang. Traffic Flow Prediction With Big Data: A Deep Learning Approach. *IEEE Transactions on Intelligent Transportation Systems*, pages 1–9, 2014.
- [8] Eleni I. Vlahogianni, Matthew G. Karlaftis, and John C. Golias. Short-term traffic forecasting: Where we are and where we are going. *Transportation Research Part C: Emerging Technologies*, 43:3–19, 6 2014.
- [9] Rui Fu, Zuo Zhang, and Li Li. Using LSTM and GRU neural network methods for traffic flow prediction. In *2016 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, pages 324–328. IEEE, 11 2016. ISBN 978-1-5090-4423-8.
- [10] S. Suhas, V. Vismaya Kalyan, Manoj Katti, B. V. Ajay Prakash, and C. Naveena. A Comprehensive Review on Traffic Prediction for Intelligent Transport System. In *2017 International Conference on Recent Advances in Electronics and Communication Technology*, pages 138–143. IEEE, 3 2017. ISBN 978-1-5090-6701-5.
- [11] Rohan More, Abhishek Mugal, Sheetal Rajgure, Rahul B. Adhao, and V K. Pachghare. Road traffic prediction and congestion control using Artificial Neural Networks. In *2016 International Conference on Computing, Analytics and Security Trends (CAST)*, pages 52–57. IEEE, 12 2016. ISBN 978-1-5090-1338-8.
- [12] Michael I. Jordan. Serial Order: A Parallel Distributed Processing Approach. *Advances in Psychology*, 121:471–495, 1 1997.
- [13] Zhenyi Xu, Yu Kang, Yang Cao, and Longchuan Yue. Residual Autoencoder-LSTM for City Region Vehicle Emission Pollution Prediction. In *2018 IEEE 14th International Conference on Control and Automation*, pages 811–816. IEEE, 6 2018. ISBN 978-1-5386-6089-8.
- [14] Felix A. Gers, Jrgen Schmidhuber, and Fred Cummins. Learning to Forget: Continual Prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 10 2000.
- [15] A. Graves, M. Liwicki, S. Fernandez, R. Bertolami, H. Bunke, and J. Schmidhuber. A Novel Connectionist System for Unconstrained Handwriting Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(5):855–868, 5 2009.
- [16] A. Graves and J. Schmidhuber. Framewise phoneme classification with bidirectional LSTM networks. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, volume 4, pages 2047–2052. IEEE. ISBN 0-7803-9048-2.
- [17] Sepp Hochreiter and Jrgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 11 1997.
- [18] Klaus Greff, Rupesh Kumar Srivastava, Jan Koutník, Bas R. Steunebrink, and Jrgen Schmidhuber. LSTM: A Search Space Odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222 – 2232, 3 2017.
- [19] Zheng Zhao, Weihai Chen, Xingming Wu, Peter C. Y. Chen, and Jingmeng Liu. LSTM network: a deep learning approach for short-term traffic forecast. *IET Intelligent Transport Systems*, 11(2):68–75, 3 2017.
- [20] Heng Shi, Minghao Xu, and Ran Li. Deep Learning for Household Load Forecasting A Novel Pooling Deep RNN. *IEEE Transactions on Smart Grid*, 9(5):5271–5280, 9 2018.
- [21] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. 6 2014.
- [22] Da Zhang and Mansur R. Kabuka. Combining weather condition data to predict traffic flow: a GRU-based deep learning approach. *IET Intelligent Transport Systems*, 12(7):578–585, 9 2018.
- [23] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *3rd International Conference for Learning Representations*, San Diego, 12 2014.
- [24] Zhiqiang Zhang, Yi Zhao, Xiangke Liao, Wenqiang Shi, Kenli Li, Quan Zou, and Shaoliang Peng. Deep learning in omics: a survey and guideline. *Briefings in Functional Genomics*, 9 2018.